❹ If it doesn't exist, returns a 404 Not Found result and short-circuits the pipeline

As before, you've derived from `ActionFilterAttribute` for simplicity and overridden the `OnActionExecuting` method. The main functionality of the filter relies on the `DoesRecipeExist()` method of `RecipeService`, so the first step is to obtain an instance of `RecipeService`. The `context` parameter provides access to the `HttpContext` for the request, which in turn lets you access the DI container and use `RequestServices.GetService()` to return an instance of `RecipeService`.

WARNING This technique for obtaining dependencies is known as *service location* and is generally considered to be an antipattern. In section 22.3 I'll show you a better way to use the DI container to inject dependencies into your filters.

As well as `RecipeService`, the other piece of information you need is the `id` argument of the `Get` and `Edit` action methods. In action filters, model binding has already occurred, so the arguments that the framework will use to execute the action method are already known and are exposed on `context.ActionArguments`.

The action arguments are exposed as `Dictionary<string, object>`, so you can obtain the `id` parameter using the `"id"` `string` key. Remember to cast the object to the correct type.

TIP Whenever I see magic strings like this, I always try to replace them by using the `nameof` operator. Unfortunately, `nameof` often won't work for method arguments like this, so be careful when refactoring your code. I